

Examen

Examen Parcial de Algoritmos y Estructuras de Datos

Universidad Blas Pascal

Alumno: Gonzalo Ramiro Alvarez Campos

Aclaración

Cada parte de los ejercicios va a quedar en un PR por separado para que sea más fácil evaluar.

El planteo de branches va a ser el siguiente: los puntos del examen se desarrollan en orden, cada nuevo punto tiene su propio branch y un PR que apunta al punto anterior. De esa manera cada PR refleja un avance concreto del desarrollo.

Para ver todo el examen completo hay que ir a la branch p2-ejercicio3, que contiene el código final.

El video explicativo se grabó con Loom mientras se hace una demo del programa. El link al video se va a subir a este mismo README.

Plataforma de análisis de incidentes y rutas Parte 1

Descripción general

El sistema gestiona eventos/urgencias acorde a prioridad, categoría, origen y destino. La idea es poder crear un sistema que sea capaz de hacer un triage de los eventos y de gestionar el orden en el cual se procesan.

Estructura del proyecto

```
Examen/
├── main.py                # archivo principal (orquestador)
├── models/
│   └── event.py           # modelo de los eventos: define la estructura
│                           # y características que tiene un evento
├── storage/
│   ├── event_store.py    # almacenamiento de los eventos
│   └── index.py          # gestión de la búsqueda de eventos
├── analisis/
│   └── text_analyzer.py  # análisis de textos de los distintos eventos
├── queues/
│   ├── priority_queue.py # cola de prioridad (wrapper sobre heapq)
│   └── incident_queue.py # cola FIFO de incidentes (wrapper sobre
deque)
└── router/
    └── router.py          # gestión de rutas (a desarrollar como grafo
                           # en la parte 2)
```

Decisiones de diseño

¿Por qué estas 5 clases?

Estas clases son los pilares de la app: permiten el manejo dinámico de variables basadas en modelos mutables, heredables y manipulables, pero con fundamentos bien definidos.

Responsabilidades de cada clase

- **Event**: define la estructura y métodos de los eventos.
- **EventStore**: define la estructura y capacidades de almacenamiento.
- **Index**: define los métodos de búsqueda para eventos.
- **TextAnalyzer**: define los métodos para procesar textos.
- **Router**: gestiona las rutas y define el grafo.

Relaciones entre clases

La clase `Event` es probablemente la más usada, ya que define los eventos: la variable base de todo el programa.

¿Por qué modularicé en carpetas?

Esta decisión es la más debatible: por lo general no hay una forma 100% correcta de estructurar un proyecto, depende mucho de los gustos y necesidades al momento de crearlo. Aquí modularicé para tener una separación de responsabilidades, lo que hace más fácil auditar el código y trabajar en él. La estructura tiene cierto parecido con el patrón MVC que se usa en el backend para el manejo de datos, donde tenemos una *source of truth* en los modelos, que son usados por los otros módulos y se muestran en `main` (en este caso).

Ejercicio 2

Estructuras elegidas

- Para la `PriorityQueue` se utilizó **heap** (vía `heapq`) dado que es extremadamente fácil organizar prioridades con un sistema numérico donde el de mayor prioridad es el número más bajo (min-heap). Es un ordenamiento casi natural con poco código.
- El heap permite un ordenamiento natural a través de un árbol binario autobalanceado, donde al sacar el elemento raíz (el próximo en el orden de prioridad), reacomodar la colección es relativamente sencillo y eficiente.
- Para la `IncidentQueue` se utilizó **deque** ya que una lista simple es muy ineficiente: `list.pop(0)` es $O(n)$ porque tiene que correr todos los elementos un lugar a la izquierda. `deque` permite sacar elementos desde la izquierda con `popleft()` en $O(1)$, lo cual es extremadamente eficiente. Esto hace que la cola de incidentes pueda ser procesada rápidamente aun con grandes volúmenes de datos. Se le pueden dar distintos usos como pre-triage o cola de notificaciones: es una cola FIFO que sirve para acompañar a la `PriorityQueue`.

Complejidades

Las complejidades fueron explicadas en cada operación con docstrings, pero en resumen:

Estructura	Operación	Complejidad
PriorityQueue (heap)	peek	O(1)
PriorityQueue (heap)	add / pop	O(log n)
PriorityQueue (heap)	is_empty / size	O(1)
IncidentQueue (deque)	add_incident / get_next_incident	O(1)
IncidentQueue (deque)	is_empty / size	O(1)

Se priorizó eficiencia haciendo un análisis de la *Big-O notation*: O(1) para las lecturas rápidas (raíz del min-heap, ambos extremos del deque) y O(log n) para inserción/extracción en el heap.

Ejercicio 3

Algoritmos elegidos: Bubble Sort ($O(n^2)$) y Merge Sort ($O(n \log n)$). Se eligió uno simple y uno eficiente para que la diferencia de comportamiento se note al crecer el tamaño de los datos.

Búsquedas comparadas: búsqueda secuencial ($O(n)$), búsqueda binaria propia ($O(\log n)$) y bisect del módulo estándar.

Baseline: `sorted()` de Python (Timsort, implementado en C).

Resultados (medidos con *timeit* y *tracemalloc*)

Tiempo de ejecución (segundos)

n	binary_search	sequential_search	bisect_search	bubble_sort	merge_sort	sorted (Timsort)
100	0.000017	0.000004	0.000014	0.000397	0.000138	0.000005
500	0.000068	0.000063	0.000053	0.047075	0.000901	0.000023
1.000	0.000120	0.000158	0.000115	0.358785	0.004042	0.000055

Memoria pico (MB)

n	binary_search	sequential_search	bisect_search	bubble_sort	merge_sort	sorted (Timsort)
100	0.001032	0.000585	0.001160	0.000681	0.002032	0.001433
500	0.012336	0.000633	0.012336	0.000633	0.008536	0.008384
1.000	0.024320	0.000633	0.024320	0.000633	0.016456	0.016384

Análisis

Bubble Sort se dispara: su complejidad es cuadrática $O(n^2)$, así que cuanto más grande es el arreglo,

peor es la performance. Recorre la lista una y otra vez moviendo elementos constantemente, lo cual es extremadamente costoso. Pasar de 100 a 1.000 elementos lo hace ~900x más lento.

Merge Sort se degrada mucho menos rápido porque es **$O(n \log n)$** : divide el arreglo en mitades sucesivamente y luego las une ordenadas. Es la opción razonable para arreglos grandes.

La desventaja de Merge Sort es la **memoria adicional** que requiere para hacer esas particiones (se ve en la tabla: ~16 MB pico contra los ~0 MB de Bubble Sort, que es *in-place*). Es el clásico **trade-off tiempo vs memoria**: Merge sacrifica memoria para ganar tiempo.

`sorted()` (Timsort de Python) le gana a ambos por amplio margen porque está implementado en C y combina Merge Sort con Insertion Sort, optimizado para datos del mundo real.

En cuanto a búsquedas: Sequential es la más simple (no requiere ordenar), pero crece linealmente. Binary y Bisect son mucho más rápidas en listas grandes, pero tienen un costo "oculto": **necesitan la lista ordenada**, y en estos benchmarks el `sorted()` previo se incluye en la medición — por eso aparecen con memoria alta. Esto plantea un trade-off práctico: si vamos a buscar **una sola vez**, conviene secuencial. Si vamos a buscar **muchas veces**, conviene ordenar una vez y usar binaria.

Ejercicio 4

Análisis

Se optó por un `dict` (diccionario de Python) como índice. Implementar un hashmap a mano sería reinventar la rueda: Python ya lo trae implementado en C, con muy buena performance para búsquedas por clave ($O(1)$ promedio).

Se utilizó `id` para indexar el diccionario ya que es un valor único, lo cual es vital para trabajar con diccionarios. En un trabajo real seguramente se utilizaría un UUID y no un `id` numerado linealmente como hacemos aquí, pero para fines didácticos sirve.

Sobre colisiones

Una **colisión** ocurre cuando dos claves *distintas* producen el mismo hash (o caen en el mismo bucket de la tabla después del módulo). No es lo mismo que insertar dos veces la misma clave: eso simplemente sobrescribe el valor.

Las dos estrategias clásicas para resolverlas son:

- **Encadenamiento (separate chaining)**: cada bucket guarda una lista de pares (`clave, valor`) que comparten ese slot. Al buscar, se recorre la lista del bucket comparando claves.
- **Direccionamiento abierto (open addressing)**: si el bucket está ocupado, se busca el siguiente disponible siguiendo alguna regla (linear probing, quadratic probing, etc.).

Python usa una variante de direccionamiento abierto en su implementación de `dict`, y redimensiona automáticamente la tabla cuando se llena para mantener la performance. Como usuario no me tengo que preocupar por la función hash ni por las colisiones: el lenguaje las resuelve transparentemente, y por eso elegí no implementar una tabla hash propia.

Ejercicio 5 — Síntesis integradora

a) Decisiones que mejoraron el rendimiento

El uso de **heap** y **deque** mejora ampliamente la performance del programa: son estructuras altamente eficientes y no reinventan la rueda, sino que utilizan herramientas ya testeadas y con soporte del ecosistema de Python. Por ejemplo, `deque.popleft()` es $O(1)$ mientras que `list.pop(0)` es $O(n)$ — y la diferencia se nota apenas hay un volumen razonable de datos.

El uso de `dict` para el `Index` permite un acceso rápido y cómodo a la información (búsqueda $O(1)$ promedio), además de ser un favorito personal y un tipo de dato con el cual tengo mucha experiencia.

A nivel de algoritmos, la elección de **Merge Sort sobre Bubble Sort** marcó la diferencia más cuantificable del trabajo: para 1.000 elementos, Bubble tarda ~ 0.36 s mientras que Merge tarda ~ 0.004 s. Es la misma tarea (ordenar) pero con dos órdenes de magnitud de diferencia, sólo por elegir bien el algoritmo.

b) Trade-off tiempo vs memoria

Al elegir entre algoritmos de sorting se nota claramente cómo el Big-O es una herramienta vital para analizar **preventivamente** los algoritmos disponibles. Antes de tipear, ya podemos saber cuál va a escalar y cuál no, y de paso evitar dolores de cabeza. También aprendemos que muchas veces hay que hacer concesiones: **tiempo vs memoria**.

Dónde aparece este trade-off en el sistema:

- **Merge Sort:** ganamos tiempo ($O(n \log n)$) pero pagamos memoria ($O(n)$ extra para las particiones).
- **Index (dict):** ganamos tiempo de búsqueda ($O(1)$ promedio) pero pagamos memoria (un slot por clave en la tabla hash).
- **Búsqueda binaria:** ganamos tiempo por consulta ($O(\log n)$) pero pagamos el costo previo de mantener la lista ordenada ($O(n \log n)$ inicial + $O(n)$ en memoria).

En todos los casos, la decisión depende del **patrón de uso real**: si vamos a buscar muchas veces, vale la pena pagar la memoria del índice; si es una búsqueda única, no.

c) POO + modularidad + estructuras + medición → software mantenible

Todo decanta en la **escalabilidad** y la **mantenibilidad**. Cualquiera puede hacer un algoritmo básico que corra a tiempos decentes, pero crear un programa que maneje cientos de miles de requests y miles de eventos es otro mundo. Estructurar correctamente el programa, elegir con criterio las estructuras de datos, modularizar de forma ordenada y saber medir la eficiencia, permite construir un sistema que resista el paso del tiempo y los aumentos de requisitos (algo muy común en la vida real).

Cada uno de estos cuatro elementos aporta algo distinto:

- **POO:** encapsula la complejidad. Quien usa la clase no necesita saber qué hay debajo. Por ejemplo, `priority_queue.add(event)` no expone el `heapq` ni la tupla `(priority, counter, event)` interna.

- **Modularidad:** permite cambiar implementaciones sin tocar el resto del sistema. Si mañana se decide reemplazar el `dict` del `Index` por una tabla hash propia, sólo se modifica `storage/index.py`.
- **Estructuras:** el rendimiento real depende de qué hay adentro. La misma API puede ser $O(1)$ o $O(n)$ según la estructura elegida.
- **Medición:** sin benchmark, las decisiones serían intuición. Con `timeit` y `tracemalloc` (punto 3), las decisiones se justifican con datos.

Sobre mantenibilidad puntualmente: un código bien estructurado es un código fácil de mantener, y lo digo por mucha experiencia propia. He visto código legacy tan mal estructurado que hacer el más simple de los cambios puede convertirse en un dolor de cabeza enorme — y es exactamente lo que la separación por responsabilidades busca evitar.

Plataforma de análisis de incidentes y rutas Parte 2

Descripción general (Parte 2)

La segunda parte extiende el sistema de la Parte 1 con **estructuras avanzadas, grafos y algoritmos avanzados**. La idea es que el `Router` (que en la Parte 1 quedó como esqueleto) se convierta en el corazón del análisis de la red origen → destino, y que se sumen capacidades de búsqueda de patrones en texto y manejo de información sensible (RSA demostrativo).

Estructura del proyecto (actualizada para Parte 2)

```
Examen/
├── main.py                # archivo principal (orquestador)
├── benchmark.py           # script de benchmarks (Parte 1)
├── models/
│   └── event.py           # modelo de los eventos
├── storage/
│   ├── event_store.py     # almacenamiento de los eventos
│   └── index.py           # índice por id (dict – Parte 1)
├── analisis/
│   └── text_analyzer.py   # análisis de textos de los eventos
├── queues/
│   ├── priority_queue.py  # cola de prioridad (heapq)
│   └── incident_queue.py  # cola FIFO de incidentes (deque)
├── trees/
│   └── union_find.py       # Union-Find con path compression
│                           # y union by rank (Parte 2 - punto 1)
├── graphs/
│   └── graph.py            # grafo no dirigido y ponderado:
│                           # BFS, DFS, Dijkstra, Kruskal MST
│                           # (Parte 2 - punto 2)
├── crypto/
│   └── rsa.py              # RSA educativo: gcd, extended_gcd,
│                           # mod_inverse, clase RSA
│                           # (Parte 2 - punto 3)
├── router/
│   └── router.py           # red de rutas: usa Union-Find para
│                           # zonas conectadas (Parte 2 - punto 1)
└── utils/
    └── utils.py            # decoradores de medición y helpers
```

Decisiones de diseño (Parte 2)

Ejercicio 1

Opción elegida: B — Union-Find con compresión de caminos + unión por rango.

¿Por qué Union-Find?

La consigna del caso menciona "analizar red de rutas (grafos) y optimizar decisiones". Una pregunta natural en una red de incidentes es: "**¿estos dos sistemas están conectados (directa o indirectamente) por alguna ruta?**". Union-Find responde esto en $O(\alpha(n)) \approx O(1)$ amortizado, sin necesidad de recorrer el grafo entero cada vez.

Otras razones por las que es apropiada:

- **Implementación corta** y autocontenida.
- **Se conecta naturalmente con grafos:** el algoritmo de Kruskal (árbol de expansión mínima del punto 2) usa Union-Find internamente.
- **Aplicación concreta al caso:** agrupar zonas conectadas a partir de los pares (origen, destino) de los incidentes.

Optimizaciones aplicadas

- **Compresión de caminos (find recursivo con asignación):** cada vez que se busca la raíz, se aplasta el árbol haciendo que cada nodo del camino apunte directo a la raíz. La próxima búsqueda es $O(1)$.
- **Unión por rango:** al unir dos árboles, el más bajo cuelga del más alto. Esto evita que los árboles degeneren en listas y mantiene la altura logarítmica.

Combinando ambas, las operaciones `find`, `union` y `connected` son $O(\alpha(n)) \approx O(1)$ amortizado, donde α es la inversa de la función de Ackermann (crece tan lento que para cualquier n realista, $\alpha(n) \leq 4$).

Aplicación al sistema (Router)

El Router que en la Parte 1 era un esqueleto ahora usa el `UnionFind` para mantener las **zonas conectadas** de la red. Cada vez que llega un incidente con (origen, destino), `add_route` los registra como nodos y los une en la misma zona. Después se puede preguntar:

- `are_connected(a, b)` → ¿están en la misma zona? $O(\alpha(n))$.
- `zone_count()` → ¿cuántas zonas disjuntas hay? $O(1)$.

Para soportar incorporación dinámica de nodos (los orígenes/destinos no se conocen de antemano), se extendió `UnionFind` con un método `add()` que agrega un elemento nuevo en $O(1)$.

Limitaciones conocidas

- Union-Find **no permite "deshacer" uniones fácilmente:** si una ruta se cae y queremos desconectar una zona, hay que reconstruir desde cero. Para este caso no es problema porque los incidentes registrados no se "borran".

- No reemplaza un grafo completo: no permite obtener el camino concreto entre dos nodos ni medir distancias. Para eso vamos a necesitar un grafo con BFS/Dijkstra (punto 2).

Complejidades

Operación	Complejidad
<code>UnionFind.find</code>	$O(\alpha(n)) \approx O(1)$
<code>UnionFind.union</code>	$O(\alpha(n)) \approx O(1)$
<code>UnionFind.connected</code>	$O(\alpha(n)) \approx O(1)$
<code>UnionFind.num_sets</code>	$O(1)$
<code>UnionFind.add</code>	$O(1)$
<code>Router.add_route</code>	$O(\alpha(n))$
<code>Router.are_connected</code>	$O(\alpha(n))$
<code>Router.zone_count</code>	$O(1)$

Ejercicio 2

Modelado del grafo

La red origen → destino se modeló como un **grafo no dirigido y ponderado**:

- **No dirigido**: una ruta entre A y B funciona en ambos sentidos. Internamente, `add_edge(A, B, w)` agrega la arista en las dos listas de adyacencia.
- **Ponderado**: cada arista tiene un peso (latencia simulada). Sin pesos, Dijkstra y Kruskal pierden sentido.
- **Lista de adyacencia**: `dict` donde cada nodo mapea a una lista de tuplas (`vecino, peso`). Memoria $O(V + E)$, apropiada para grafos dispersos como esta red.

Algoritmos implementados

- **BFS (Breadth-First Search)**: exploración por niveles desde un nodo. Usa una cola FIFO (deque). Sirve para alcance y, sin pesos, para camino más corto en número de aristas.
- **DFS (Depth-First Search)**: exploración profundizando antes de volver. Implementado de forma recursiva con un helper `_dfs_visit`. Sirve para detectar ciclos, componentes conectadas y exploraciones exhaustivas.
- **Dijkstra (shortest_distances)**: distancia mínima desde un nodo a todos los demás, sobre un grafo con pesos no negativos. Usa `heapq` como cola de prioridad. El truco está en que un mismo nodo puede entrar varias veces a la PQ con distintas distancias; al sacarlo se descartan las versiones "viejas" con `if current_dist > distances[current]: continue`.
- **Kruskal (minimum_spanning_tree)**: árbol de expansión mínima. Ordena las aristas por peso y las va agregando al MST si no forman ciclo. La detección de ciclos se hace con el `UnionFind` del Ejercicio 1, lo cual conecta directamente los dos puntos de esta parte.

Aplicación al caso

- **BFS / DFS** → "¿qué sistemas son alcanzables desde X?".
- **Dijkstra** → "¿cuál es la ruta de menor latencia entre dos sistemas?".
- **Kruskal MST** → "¿cuál es el conjunto mínimo de rutas para mantener todos los centros conectados?".

Complejidades

Operación	Complejidad	Notas
<code>Graph.add_node</code>	$O(1)$ promedio	acceso a <code>dict</code>
<code>Graph.add_edge</code>	$O(1)$ promedio	append en ambas listas
<code>Graph.neighbors</code>	$O(1)$ promedio	acceso a <code>dict</code>
<code>Graph.bfs</code>	$O(V + E)$	cada nodo y arista visitados 1 vez
<code>Graph.dfs</code>	$O(V + E)$	igual que BFS
<code>Graph.shortest_distances</code>	$O((V + E) \log V)$	con <code>heapq</code> como PQ
<code>Graph.minimum_spanning_tree</code>	$O(E \log E)$	dominado por el sort de aristas

Limitaciones conocidas

- **Multi-aristas y self-loops:** el grafo permite agregar varias aristas entre los mismos nodos y aristas que conectan un nodo consigo mismo. Para este caso no fue problema, pero si se quisiera grafo simple habría que filtrar en `add_edge`.
- **Pesos negativos:** Dijkstra **no funciona** con pesos negativos. Para esos casos se necesitaría Bellman-Ford. En el caso del examen las latencias son siempre positivas.
- **Grafo desconectado:** si hay zonas no alcanzables desde el `start`, Dijkstra las devuelve con distancia `float('inf')`. Kruskal en ese caso devolvería un *bosque* de expansión, no un único árbol.

Mediciones

Resultados de `benchmark.py` sobre grafos contruidos a partir de eventos (cadena lineal con pesos aleatorios entre 1 y 100):

Tiempo (segundos)

n	BFS	DFS	Dijkstra	Kruskal
51	0.000042	0.000043	0.000128	0.000121
201	0.000123	0.000223	0.000450	0.000346
501	0.000342	0.000827	0.001135	0.001076

Memoria pico (MB)

n	BFS	DFS	Dijkstra	Kruskal
51	0.003888	0.005128	0.003641	0.010304
201	0.011872	0.019912	0.014265	0.034216
501	0.044448	0.061192	0.029169	0.080928

Análisis breve

- **BFS / DFS** crecen aproximadamente lineal con n ($O(V + E)$). DFS recursivo gasta un poco más de memoria por la pila de llamadas.
- **Dijkstra** es $\sim 2\text{-}3\times$ más lento que BFS porque cada `heappush/heappop` cuesta $O(\log V)$ y se hace varias veces por nodo.
- **Kruskal** tiene la memoria pico más alta porque construye una lista ordenada de aristas y mantiene el `UnionFind` con `parent` y `rank` en paralelo. El tiempo está dominado por el sort: $O(E \log E)$.
- En todos los casos la diferencia con la cadena de eventos del ejemplo ($V \approx E + 1$) es chica; en grafos densos reales Kruskal y Dijkstra empiezan a despegarse mucho de BFS/DFS.

Ejercicio 3

Opciones elegidas: dos de tres permitidas:

1. **Análisis de texto:** búsqueda de patrones con **fuerza bruta vs KMP** (Knuth-Morris-Pratt).
2. **Teoría de números:** **RSA demostrativo** (cifrar/descifrar mensaje corto).

Búsqueda de patrones (fuerza bruta vs KMP)

¿Qué problema resuelve?

Dado un texto largo y un patrón (palabra o secuencia), encontrar **dónde aparece el patrón** dentro del texto. Aplicación al caso: buscar palabras clave (ej: "incendio", "caída") dentro de las descripciones de incidentes para disparar alertas.

Ambos métodos están implementados como métodos de la clase `TextAnalyzer`:

- `brute_force_search(text, pattern)` — comparación naive en cada posición. **$O(n \cdot m)$** .
- `kmp_search(text, pattern)` — usa la tabla LPS (Longest Prefix Suffix) precomputada para no retroceder en el texto. **$O(n + m)$** .

Cómo se diferencian

- **Fuerza bruta:** cuando hay un mismatch, retrocede el índice del texto y empieza a comparar desde la siguiente posición. En textos repetitivos, hace mucho trabajo redundante.
- **KMP:** el índice del texto **nunca retrocede**. Cuando hay mismatch, usa la tabla LPS del patrón para saber cuántos caracteres del patrón sí siguen siendo válidos, y reanuda desde ahí.

Mediciones

Para que la diferencia se note, se usó un caso patológico (texto repetitivo "aaaa . . . aaab" y patrón "aaaaa . . . aaab"):

n	Brute Force (s)	KMP (s)	Speedup
1000	0.016893	0.000404	~42x
10000	0.222571	0.004399	~50x
50000	1.220592	0.024399	~50x

Análisis breve

- **Fuerza bruta crece cuadráticamente con n:** pasar de 10.000 a 50.000 caracteres hace que el tiempo se quintuplique con factor extra (multiplica por ~5.5x), tendencia coherente con $O(n \cdot m)$.
- **KMP crece lineal:** cada 10x en n da aproximadamente 10x más tiempo, coherente con $O(n + m)$.
- **El precomputo de LPS es el "costo único" que paga KMP ($O(m)$).** En patrones cortos sobre textos largos es casi gratis comparado con el ahorro al recorrer el texto.
- En textos "normales" (no patológicos) la diferencia es chica. KMP brilla en textos repetitivos o con muchos matches parciales.

RSA demostrativo

¿Qué problema resuelve?

Permite **cifrar información sensible** (por ejemplo, descripciones de incidentes con datos de sistemas afectados) de manera que solo quien tenga la **clave privada** pueda descifrarla. Aplicación al caso: la consigna del enunciado menciona "resguardar información sensible mediante encriptación (RSA a nivel demostrativo)".

Cómo funciona

1. Se eligen dos primos p y q .
2. Se calcula $n = p * q$ (módulo) y $\phi(n) = (p-1) * (q-1)$ (función totiente de Euler).
3. Se elige un exponente público e coprimo con $\phi(n)$ (típicamente 65537).
4. Se calcula el exponente privado d tal que $(d * e) \bmod \phi(n) == 1$. Para esto se usa el **algoritmo extendido de Euclides**.
5. **Clave pública:** (n, e) . **Clave privada:** (n, d) .

Operaciones:

- **Cifrar:** $c = m^e \bmod n$.
- **Descifrar:** $m = c^d \bmod n$.

La implementación vive en `crypto/rsa.py` y expone:

- `gcd(a, b)` — máximo común divisor (Euclides).
- `extended_gcd(a, b)` — versión extendida que devuelve también los coeficientes de Bézout.
- `mod_inverse(e, phi)` — inverso modular usando `extended_gcd`.
- `RSA(p, q, e=65537)` — clase con `encrypt(message)` y `decrypt(ciphertext)`. Acepta tanto enteros como strings (cifra carácter por carácter).

Demo

Con primos $p=61$, $q=53$ ($n=3233$):

```
Public key (n, e): (3233, 7)
Private key (n, d): (3233, 1783)
Encrypt int 42 -> 240; decrypt -> 42
Encrypt "incidente-A:42" -> [3020, 1544, 24, 3020]... (14 ints)
Decrypt -> "incidente-A:42"
```

Límites de seguridad por tamaños

La seguridad de RSA depende de que **factorizar n (descomponerlo en $p * q$) sea computacionalmente caro**. Esa dificultad crece exponencialmente con el tamaño de los primos.

Tamaño de n	Estado
Hasta ~64 bits	Trivial de factorizar en milisegundos. Sólo demo.
512 bits	Factorizable en horas/días con recursos modernos. Inseguro.
1024 bits	Considerado inseguro desde 2010 aprox.
2048 bits	Estándar mínimo actual para uso real.
4096 bits	Recomendado para alta seguridad / largo plazo.

Limitaciones de esta implementación

- **Primos chicos:** en el demo se usan primos del orden de decenas o centenas. Cualquiera puede factorizar $n=3233$ mentalmente.
- **Sin padding:** RSA "puro" sin padding (como OAEP) tiene vulnerabilidades conocidas (mismo mensaje siempre genera mismo cifrado, ataques por mensaje corto, etc.).
- **Cifrado carácter por carácter:** cada carácter se cifra por separado, lo cual es ineficiente y muestra patrones (caracteres iguales generan cifrados iguales). En la práctica RSA se usa para cifrar **claves simétricas cortas**, no mensajes largos.
- **Generación de primos no incluida:** el constructor recibe p y q como argumentos. Una implementación real debería generar primos grandes aleatoriamente con tests de primalidad.

Mensaje a transmitir

Esta implementación es **estrictamente didáctica**. Sirve para mostrar **cómo funciona** RSA matemáticamente (módulo, exponente público/privado, inverso modular), no para proteger información real.

Uso de herramientas de IA

Para este trabajo utilicé **Claude Code (Anthropic)** como herramienta de acompañamiento conceptual, en línea con lo permitido por la consigna del examen.

En qué me ayudó la IA

- **Explicaciones conceptuales:** cómo funciona internamente un heap, por qué `deque` es más eficiente que `list` para una cola FIFO, qué es Big-O y cómo se mide, cómo funcionan los decoradores en Python (con paralelos a JavaScript), qué hace `functools.wraps`.
- **Detección de bugs y errores:** en cada iteración le mostré el código que escribía y me devolvía feedback indicando qué fallaba y por qué (por ejemplo: usar `@property.setter` en métodos comunes, comparar objetos sin `__lt__`, mezclar tipos en búsquedas binarias, falta de `random.shuffle` antes de medir un sort, etc.).
- **Sugerencias de organización:** opciones para estructurar carpetas y módulos, cuándo conviene un wrapper sobre `heapq/deque`, alternativas para acumular datos de benchmarks (closure vs variable de módulo).
- **Correcciones de redacción:** tildes, typos y reorganización de tablas en este README.

Qué hice yo

- **Las decisiones de diseño:** estructura de carpetas, qué clase vive en cada módulo, qué responsabilidad tiene cada una, cómo se relacionan entre sí.
- **El código tipeado:** las clases del sistema (`Event`, `EventStore`, `Index`, `TextAnalyzer`, `Router`), los wrappers (`PriorityQueue`, `IncidentQueue`), los algoritmos del punto 3 (búsquedas y ordenamientos), el script de benchmark.
- **La justificación escrita:** lo que está en este README sobre por qué cada decisión (heap, deque, Bubble vs Merge, trade-offs) fue redactado por mí — la IA pulió forma y corrigió typos, pero los argumentos son míos.
- **El criterio para iterar:** cada feedback de la IA fue una sugerencia. Yo decidía qué aplicar, qué descartar y cómo aplicarlo.

Aclaración explícita

Esta declaración fue escrita por mí. El contenido de este README, incluyendo esta misma sección, refleja mi propio criterio sobre el uso que hice de la herramienta. La IA cumplió un rol de **tutoría iterativa**, no de resolución automática del examen.

Anexo — Diálogos con IA

A continuación, una selección de diálogos representativos del proceso, mostrando preguntas concretas, fragmentos relevantes de respuesta y cómo cada uno aportó al trabajo.

Diálogo 1 — Por dónde empezar el punto 1 (POO + modularidad)

Pregunta: ¿Qué pide exactamente el punto 1 del examen y qué clases tengo que crear?

Fragmento relevante de la respuesta:

Te están dando los nombres de las clases ya elegidos: `Event`, `EventStore`, `Index`, `Router`, `TextAnalyzer`. Cada una tiene una responsabilidad clara dentro del caso:

- `Event` → representa un incidente. Es el "objeto de dominio".
- `EventStore` → almacena la colección de eventos.
- `Index` → estructura de acceso rápido por clave (se conecta con el punto 4).
- `Router` → maneja la red de rutas (origen → destino).
- `TextAnalyzer` → busca patrones en el texto de los eventos.

No te pide implementar todo el sistema funcionando. Te pide diseñar la arquitectura.

Cómo me ayudó: entendí que el punto 1 era de **diseño**, no de implementación completa. Me ahorró tiempo enfocándome en la separación de responsabilidades antes de tipear código.

Diálogo 2 — Cómo funciona internamente un heap

Pregunta: ¿Podés explicarme cómo funciona un heap antes de usar `heapq`?

Fragmento relevante de la respuesta:

Un heap es un árbol binario con dos reglas:

1. Está casi lleno (se completa por niveles, sin huecos).
2. Propiedad de heap: cada padre cumple una relación de orden con sus hijos. En min-heap, el padre siempre es $\leq$ que sus hijos, así que la raíz es el mínimo.

El truco está en que se guarda en una lista, no en un árbol con punteros: para el elemento en posición i , hijo izquierdo = $2 * i + 1$, hijo derecho = $2 * i + 2$, padre = $(i - 1) // 2$. Insertar y extraer son $O(\log n)$ porque la altura del árbol es $\log n$.

Cómo me ayudó: entendí **por qué** un heap es eficiente para una `PriorityQueue` (no porque sí, sino porque la raíz está siempre disponible en $O(1)$ y las inserciones/extracciones son logarítmicas). Pude defender la decisión en el README en vez de copiar la elección.

Diálogo 3 — Detección de bug en PriorityQueue

Pregunta: Acabo de hacer la PriorityQueue, ¿podés revisarla?

Fragmento relevante de la respuesta:

Bug crítico: `@add.setter` y `@get.getter` están mal usados.

```
@add.setter
def add(self, item, priority):
    ...
```

Esto **no compila** — vas a tener `NameError: name 'add' is not defined`. Los decoradores `@property/@x.setter/@x.getter` son para crear **propiedades** (atributos calculados), **no métodos normales**. Sacá los decoradores. Que sean métodos normales.

△ También falta el contador para desempates. Si dos eventos tienen la misma prioridad, Python va a comparar el `Event` y como no tiene `__lt__`, va a explotar con `TypeError`.

Cómo me ayudó: la IA detectó dos bugs que yo no había visto: uno de sintaxis (decoradores mal usados) y uno conceptual (Python no sabe comparar dos `Event` cuando empata la prioridad). Me llevó a usar la tupla (`priority, counter, item`) como clave del heap.

Diálogo 4 — Decoradores en Python (paralelo con JavaScript)

Pregunta: Me pierdo con los decoradores. Yo vengo de JavaScript, ¿es lo mismo que envolver una función pasándola como argumento? ¿Cómo haría en JS?

Fragmento relevante de la respuesta:

Es exactamente lo mismo que un Higher-Order Function en JS.

En Python:

```
@measure_time
def bubble_sort(arr): ...
```

es equivalente a:

```
def bubble_sort(arr): ...
bubble_sort = measure_time(bubble_sort)
```

El `@` es solo azúcar sintáctica. No hace nada mágico, solo te ahorra la reasignación. Cuando dudes, traducí mentalmente `@decorator` a `fn = decorator(fn)`.

Cómo me ayudó: sacó la magia del decorador y lo volvió un concepto familiar. A partir de ahí pude diseñar `measure_time` y `measure_memory` con confianza, y entender por qué hacía falta `@functools.wraps` cuando los apilaba.

Diálogo 5 — Diferencia entre "misma clave" y "colisión" en hashing

Pregunta: Mostré mi redacción del punto 4 sobre hashing y colisiones.

Fragmento relevante de la respuesta:

Decís: "La colisión ocurre cuando tenemos la misma key para un hashmap". **Esto es incorrecto.** La misma key para un hashmap **no es una colisión** — es simplemente sobrescribir el valor.

La colisión real ocurre cuando dos **keys distintas** producen el **mismo hash** (o el mismo bucket después del módulo). Por ejemplo, las keys "hola" y "perro" podrían tener hashes distintos, pero al hacer $\text{hash} \% \text{tamaño_tabla}$ caer en el mismo slot.

Cómo me ayudó: corrigió un error conceptual importante en mi entrega original. El docente probablemente lo habría flagueado: "explicar colisiones" es una de las consignas literales del punto 4, así que tenerlo bien definido era crítico. Reescribí la sección con la definición correcta.

Declaración de autoría

Las decisiones de diseño, el código tipeado y los argumentos escritos en este README son **propios**. La IA cumplió un rol de tutoría conceptual y revisión iterativa, en línea con el uso permitido por la consigna del examen.